PRESIDENCY UNIVERSITY

Itgalpura, Rajanukunte, Yelahanka Bengaluru-560064

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING AND INFORMATION SCIENCE

Department of Computer Science & Engineering

A Lab Report on

CSE3001 - Artificial Intelligence & Machine Learning

Name of the student

Name: Akash M P

Roll No: 20221LCS0021

Dr. Mahalakshmi R

Name of Faculty Signature of Faculty

TABLE OF CONTENTS

SI. NO	NAME OF EXPERIMENT	DATE	PAGE NO	MARKS	SIGNATURE
1	A REVIEW OF PYTHON PROGRAMMING				
2	INTRODUCTION TO NUMPY, PANDAS				
3	REGRESSION MODELS: SIMPLE LINEAR REGRESSION				
4	DECISION TREE CLASSIFIERS				
5	OPTIMIZATION TECHNIQUES: GRADIENT DESCENT				
6	K NEAREST NEIGHBOR CLASSIFIERS				
7	ENSEMBLE MODELS				
8	K MEANS CLUSTERING				
9	PROBABILITY THEORY				
10	FORECASTING APPLICATION				
11	RECOMMENDER SYSTEMS				

LABSHEET NO: 1	A REVIEW OF PYTHON PROGRAMMING	PAGE NO :
DATE:		

What is python?

1. Python is powerful programming language.

2. Specially used in data science, machine learning to solve daily life real-time problems and providing the solutions.

3. Python is a high-level, interpreted, Inter-active and object oriented scripting language.

Why Python?

1. Easy to understand
2. Easy syntax
3. Beginner friendly

APPLICATIONS OF PYTHON

4. Web applications
5. Desktop applications
6. Database applications
7. Web Scraping

PYTHON IDENTIFIERS:

1. A Python identifier is a name used to identify variable, function, class, module or other objects.
2. A identifier starts with a letter (a-z), (A-Z) or an underscore (_) followed by zero or more letters, and digits (0-9).

PYTHON KEYWORDS:

1. Keywords are reserved words.
2. Cannot use them as constant or variable.
3. All the keywords except "True", "False" and "None" are in lower case.
4. They are 35 keywords present in python.

```
#code to print all the keywords present in python

import keyword
print(keyword.kwlist)
a=len(keyword.kwlist)
print(a)
```

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', "continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

INDENTATION IN PYTHON:

1. In C, C++ the indentation is specified by using the flower brackets {}.

2. In python, Indentation is done by 4 white spaces or tab.

PYTHON VARIABLES

1. Variables are nothing but reserved memory locations to store values.
2. When we create a variable, automatically some space in memory is reserved.
3. Based on the data types the interpreter allocates the memory.
4. Variable can take any data type such as integer, float numbers, strings etc.

```
#exampl1:

#here 3 variables have been created and stored the values as shown below

a=50

b=50

c=a+b

print(c)
```

O/P: 100

PYTHON DATA TYPES:

1. Integer Data types
2. Float point Data types
3. Strings Data type

PYTHON DATA STRUCTURES

1. In python we have four types of data structures.

2. These data structures are very useful and important while working with advance topics such as machine learning, data science.

LIST

- List is a python data structure
- It is a combination of any data type.
- Enclosed within [] and separated by commas (,).
- List is sequence of values called items or elements.
- List is similar arrays.
- List mutable.
- List is indexible.

LIST BUILT-IN FUNCTION

Function	Meaning
len()	Returns the number of elements in a list
max()	Returns element with greater value
min()	Returns the element with lowest value
sum()	Returns the sum of all elements in the list
random.shuffle()	Shuffle the elements in a list randomly -Here we make use of a random package.

LIST OPERATORS

Operator	Meaning
"+"	Concatenates two lists with each other.
"*"	Replicates the elements of lists
"in"	-Used to determine whether the element is present in the list or not. -Returns true if the element is present in the list -Returns false if the element is not present.
"is"	Used to check whether two variables refer to the same object or not.
"del"	-Used to remove elements from the list.

LIST METHODS

Methods	Meaning
append()	Adds element at the end of the list
clear()	Clear the contents of the list
count()	Gives count of repeated items
copy()	Copy list from one list to another list
extend ()	Joins the elements of 2 lists

TUPLES

1. Tuple is another important data structures present in python.
2. Allow multiple data types.
3. Enclosed () and separated by commas (,).
4. Tuples are not mutable

5. Tuples are indexible

6. Tuples are ordered

PYTHON DICTIONARY

1. Dictionaries are used to store the values in key-value pair.
2. It's a combination of keys and values
3. Elements are accessed based on the keys.
4. It is ordered, mutable, and don't allow duplicate elements.
5. Enclosed within {} with key value pair

DICTIONARY METHODS

Methods	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of dictionary
get()	Returns the value of specified key
items()	Returns the list containing tuple pair of keys and values
keys(),values()	key()-Returns the keys of the dictionary values()-Returns the values of the dictionary

PYTHON SETS

1. Sets are used to store multiple values in one variable.
2. Sets are enclosed within {}.
3. Sets are unordered, not mutable and unidexible.
4. Sets items are unchangeable but we can remove and add new items.

SET METHODS:

Methods	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
Difference()	Returns a set containing difference between 2 or more sets
Discard()	Removes the specific item
Copy()	Returns a copy of the set

NAME: AKASH M P

ROLL NUMBER: 20221LCS0021

SECTION: 5CSE-7

In [1]:
```
import keyword
'''
Well, it's a long day today..
It sure is.
'''
print(keyword.kwlist)
print(len(keyword.kwlist))
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'clas
s', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from',
'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
'raise', 'return', 'try', 'while', 'with', 'yield']
35
```

String

In [2]:
```
a = 15
b = 5
c = a + b
print(c)
```

```
20
```

DS

Lists []

Tuples ()

Dictionaries {}

In [3]:
```
a = ([1, 2, 3, 4, 5])   # list with constructor
b = ([1.2, 3, 'hello'])
c = ['hola', 'hallo']   # list without constructor
print(a)
print(type(a))
print(b)
print(type(b))
print(c)
print(type(c))
```

```
[1, 2, 3, 4, 5]
<class 'list'>
[1.2, 3, 'hello']
<class 'list'>
['hola', 'hallo']
<class 'list'>
```

In [4]:
```
l = list(range(5, 10))
print(l)
print(l[2:4])   # slicing
print(l[0:4:2])   # slicing
```

```
[5, 6, 7, 8, 9]
[7, 8]
[5, 7]
```

Built-in Functions of list

In [5]:
```
# l = [5, 6, 7, 8, 9]
print(min(l))
print(max(l))
print(sum(l))
```

```
5
9
35
```

Random Library

In [6]:
```
import random
numbers = list(range(1, 11))
random.shuffle(numbers)
print(numbers)
```

```
[1, 4, 7, 10, 6, 3, 2, 5, 8, 9]
```

In [7]:
```
numbers = [3, 4, 10, 1, 5, 8, 2, 6, 7, 9]
numbers.append(100)
print(numbers)
numbers.clear()
print(numbers)
```

```
[3, 4, 10, 1, 5, 8, 2, 6, 7, 9, 100]
[]
```

In [8]:
```
list1 = list(range(1, 5))
list2 = list(range(5, 10))
list1.extend(list2)    # list2 extended list1
print(list1)
list2.append(list1)    # list 2 added with extended list1
print(list2)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[5, 6, 7, 8, 9, [1, 2, 3, 4, 5, 6, 7, 8, 9]]
```

In [9]:
```
num = [1, 4, 24, 1, 4, 72, 3]
print(num.index(3))   # index of 3
num.insert(2, 8)
print(num)
print(num.pop())    # last element pop
num.reverse()
print(num)
num.sort()
print(num)
num.remove(72)
print(num)
```

```
6
[1, 4, 8, 24, 1, 4, 72, 3]
3
[72, 4, 1, 24, 8, 4, 1]
[1, 1, 4, 4, 8, 24, 72]
[1, 1, 4, 4, 8, 24]
```

In [10]:
```
a = [1, 23, 4, 12]
print(12 in a)
b = [1, 3, 5, 2, 6]
```

```
print(a + b)
print(2 not in b)
```

```
True
[1, 23, 4, 12, 1, 3, 5, 2, 6]
False
```

Tuples

In [11]:
```
a = tuple(range(1, 10))
print(a)
```

```
(1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Python Dictionary

In [12]:
```
a = {
    'name': "hannah",
    'age': 20,
    'education': "Arts"
}
print(a)
print(a['education'])
print(a.keys())
print(a.values())
print(len(a))
```

```
{'name': 'hannah', 'age': 20, 'education': 'Arts'}
Arts
dict_keys(['name', 'age', 'education'])
dict_values(['hannah', 20, 'Arts'])
3
```

Built-in Methods of Dictionary

In [13]:
```
b = a.copy()
print(b)
print(b.get('age'))
```

```
{'name': 'hannah', 'age': 20, 'education': 'Arts'}
20
```

Sets

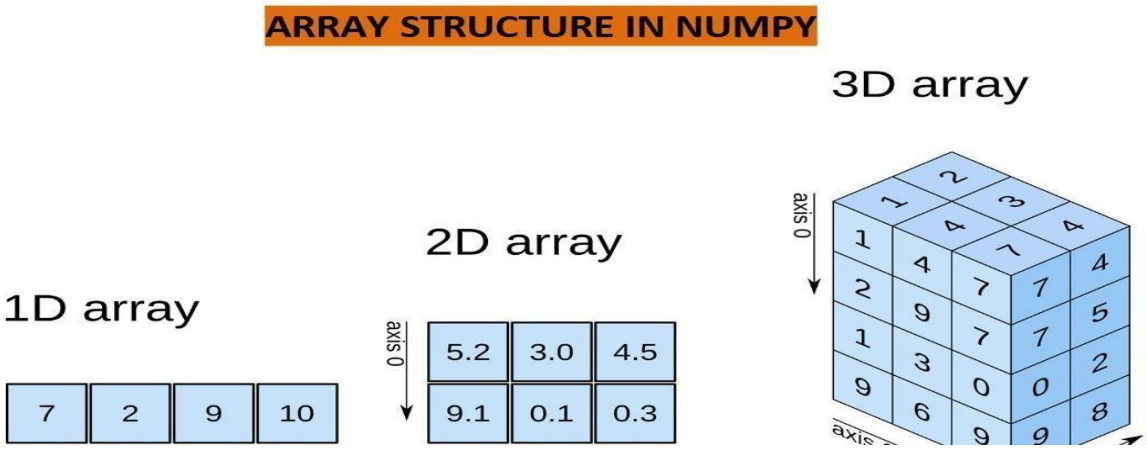
In [14]:
```
set1 = {1, 2, 3, 4}
print(set1)
set2 = {1, 23, 1, 3, 1, 2, 1}
print(set2)
print(set1.difference(set2))  # in set1 and not set2
print(set2.difference(set1))  # in set2 and not set1
set2.discard(23)
print(set2)
```

```
{1, 2, 3, 4}
{1, 2, 3, 23}
{4}
{23}
{1, 2, 3}
```

LABSHEET NO: 2	INTRODUCTION TO NUMPY, PANDAS, DICTIONARIES, DICTIONARY COMPREHENSION.	PAGE NO :
DATE:		

NUMPY ARRAYS:

1. NUMPY-Numpy stands for "Numeric Python" or "Numerical Python.
2. Numpy is a package that contains several classes, functions, variables etc. to deal with scientific calculations in Python.
3. Numpy is useful to create and process single and multi-dimensional arrays.
4. In addition, numpy contains a large library of mathematics like linear algebra functions and Fourier transformations.



Ones Array-An array in which all values are 1

```
A=np.ones(shape,dtype)
```

1. we can define the shape and data type of ones array.
2. We can create 1D, 2D & 3D ones array.
3. The default data type is float.

DATA VISUALIZATION:

1. Data visualization is graphical representation of data.
2. It involves producing images that communicate relationships among the represented data to viewers.
3. Visualizing data is an essential part of data analyzing and ML.

BAR PLOT:

1. A bar plot or bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent.
2. The bar plots can be plotted horizontally or vertically.
3. A bar chart describes the comparisons between the discrete categories. It can be created using the bar() method.

NAME: AKASH M P

ROLL NUMBER: 20221LCS0021

SECTION: 5CSE-7

In [119…
```
import numpy as np
print(np.__version__)
```

```
1.26.1
```

In [120…
```
a1 = np.array([1, 2, 3, 4, 5])
print(a1)
```

```
[1 2 3 4 5]
```

In [121…
```
print(type(a1))
```

```
<class 'numpy.ndarray'>
```

In [122…
```
a1.shape
```

Out[122…
```
(5,)
```

In [123…
```
a1.size
```

Out[123…
```
5
```

In [124…
```
# 2D array
a2 = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(a2)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [125…
```
type(a2)
```

Out[125…
```
numpy.ndarray
```

In [126…
```
a2.shape
```

Out[126…
```
(2, 4)
```

In [127…
```
a2.size
```

Out[127…
```
8
```

In [128…
```
a2.ndim
```

Out[128…
```
2
```

In [129…
```
a3 = np.array([[[1, 2, 3], [4, 5, 6], [7, 8, 9]]])
print(a3)
```

```
[[[1 2 3]
  [4 5 6]
  [7 8 9]]]
```

In [130…
```
type(a3)
```

Out[130…
```
numpy.ndarray
```

In [131…
```
a3.size
```

Out[131…
```
9
```

In [132…
```
a3.ndim
```

Out[132…
```
3
```

In [133…
```
import numpy as np
z1 = np.zeros(3)
z1
```

Out[133…
```
array([0., 0., 0.])
```

In [134…
```
z1 = np.zeros(3, dtype=int)
z1
```

Out[134…
```
array([0, 0, 0])
```

In [135…
```
z1.shape
```

Out[135…
```
(3,)
```

In [136…
```
z1.size
```

Out[136…
```
3
```

In [137…
```
z1.ndim
```

Out[137…
```
1
```

In [138…
```
type(z1)
```

Out[138…
```
numpy.ndarray
```

In [139…
```
z2 = np.zeros((3, 4))
z2
```

Out[139…
```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

In [140…
```
z2 = np.zeros((3, 4), dtype=int)
z2
```

Out[140…
```
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]])
```

In [141…
```
type(z2)
```

Out[141…
```
numpy.ndarray
```

In [142…
```
z2.shape
```

Out[142… (3, 4)

In [143…
```
z2.ndim
```

Out[143… 2

In [144…
```
z3 = np.zeros((2, 3, 4))
z3
```

Out[144…
```
array([[[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]],

       [[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]]])
```

In [145…
```
z3 = np.zeros((2, 3, 4), dtype=int)
z3
```

Out[145…
```
array([[[0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]],

       [[0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]]])
```

In [146…
```
type(z3)
```

Out[146… numpy.ndarray

In [147…
```
z3.ndim
```

Out[147… 3

In [148…
```
z3.size
```

Out[148… 24

In [149…
```
z3.shape
```

Out[149… (2, 3, 4)

In [150…
```
# 1D array
a1 = np.ones(3)
a1
```

Out[150… array([1., 1., 1.])

In [151…
```
a1 = np.ones(3, dtype=int)
a1
```

Out[151… array([1, 1, 1])

In [152…
```
type(a1)
```

Out[152… numpy.ndarray

In [153…
```
a1.ndim
```

Out[153… 1

In [154…
```
a1.shape
```

Out[154… (3,)

In [155…
```
a1.size
```

Out[155… 3

In [156…
```
a2 = np.ones([3, 4])
a2
```

Out[156…
```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

In [157…
```
type(a2)
```

Out[157… numpy.ndarray

In [158…
```
a2.ndim
```

Out[158… 2

In [159…
```
a2.size
```

Out[159… 12

In [160…
```
a2.shape
```

Out[160… (3, 4)

df = pd.read_csv('path to csv file')
df[['','']] - to get multiple column
s df['] - particular colu
mn df.sort_values(by=['col
']) df.sort_values(by=[['col', 'col'], ascending=(false, tr
ue)) df.fillna(0) - fill missing values w
ith 0 df.fillna({'Ename':'Name is missing','salary': 0.0, 'DOJ': '00-00-00'}) - fill with parti
culars df.dropna() - drop rows with missing
values

In [161…
```
import pandas as pd
df = pd.read_csv('empdata.csv')
print(df[1:4])
print(df.columns)
print(df['Empid'])
print(df[['Empid', 'Ename']])
```

```
print(df.head())
print(df.shape)
```

```
   Empid   Ename   Salary        DOJ
1   1002    Anil  23000.5  2/20/2002
2   1003  Gaurav      NaN   3/3/2002
3   1004    Hema  16500.5  9/10/2000
Index(['Empid', 'Ename', 'Salary', 'DOJ'], dtype='object')
0    1001
1    1002
2    1003
3    1004
4    1005
5    1006
Name: Empid, dtype: int64
   Empid   Ename
0   1001  Ganesh
1   1002    Anil
2   1003  Gaurav
3   1004    Hema
4   1005   Laxmi
5   1006   Anant
   Empid   Ename    Salary         DOJ
0   1001  Ganesh   1000.00  10/10/2000
1   1002    Anil  23000.50   2/20/2002
2   1003  Gaurav       NaN    3/3/2002
3   1004    Hema  16500.50   9/10/2000
4   1005   Laxmi  12000.75   10/8/2000
(6, 4)
```

Check for duplicates and remove them.

In [162…

```
df1 = pd.concat([df, df], ignore_index=True)
print(df1)
```

```
    Empid   Ename    Salary         DOJ
0    1001  Ganesh   1000.00  10/10/2000
1    1002    Anil  23000.50   2/20/2002
2    1003  Gaurav       NaN    3/3/2002
3    1004    Hema  16500.50   9/10/2000
4    1005   Laxmi  12000.75   10/8/2000
5    1006   Anant   9999.99    9/9/1999
6    1001  Ganesh   1000.00  10/10/2000
7    1002    Anil  23000.50   2/20/2002
8    1003  Gaurav       NaN    3/3/2002
9    1004    Hema  16500.50   9/10/2000
10   1005   Laxmi  12000.75   10/8/2000
11   1006   Anant   9999.99    9/9/1999
```

In [163…

```
df1.drop_duplicates()
```

Out[163…

	Empid	Ename	Salary	DOJ
0	1001	Ganesh	1000.00	10/10/2000
1	1002	Anil	23000.50	2/20/2002
2	1003	Gaurav	NaN	3/3/2002
3	1004	Hema	16500.50	9/10/2000
4	1005	Laxmi	12000.75	10/8/2000
5	1006	Anant	9999.99	9/9/1999

Capitalize column names

In [164…

```
df1.columns = [i.upper() for i in df1]  # capitalize column names   [function it
print(df1.columns)
```

```
Index(['EMPID', 'ENAME', 'SALARY', 'DOJ'], dtype='object')
```

In [165…

```
print(df.isnull())
```

```
   Empid  Ename  Salary    DOJ
0  False  False   False  False
1  False  False   False  False
2  False  False    True  False
3  False  False   False  False
4  False  False   False  False
5  False  False   False  False
```

In [166…

```
df = df.fillna({"Salary": 0.00})
df
```

Out[166…

	Empid	Ename	Salary	DOJ
0	1001	Ganesh	1000.00	10/10/2000
1	1002	Anil	23000.50	2/20/2002
2	1003	Gaurav	0.00	3/3/2002
3	1004	Hema	16500.50	9/10/2000
4	1005	Laxmi	12000.75	10/8/2000
5	1006	Anant	9999.99	9/9/1999

In [167…

```
print('Number of null values:', df.isnull().sum())
```

```
Number of null values: Empid     0
Ename     0
Salary    0
DOJ       0
dtype: int64
```

In [168…

```
df.dropna(inplace=True)      # drops rows with at least one missing value
df.dropna(axis=1, inplace=True)      # drops columns with at least one missing va
```

In [169…

```
print('Highest  Salary:',df['Salary'].max())
```

```
print('Lowest  Salary:',df['Salary'].min())
```

```
Highest Salary: 23000.5
Lowest Salary: 0.0
```

In [170…

```
print(df[df['Salary'] > 20000])
```

```
   Empid Ename   Salary        DOJ
1   1002  Anil  23000.5  2/20/2002
```

In [171…

```
print(df[['Empid', 'Ename']][df['Salary'] > 20000])
```

```
   Empid Ename
1   1002  Anil
```

In [172…

```
print(df[['Empid', 'Ename']][df['Salary'] == df['Salary'].max()])
```

```
   Empid Ename
1   1002  Anil
```

In [173…

```
print(df[['Empid', 'Ename']][df['Salary'] > df['Salary'].mean()])
```

```
   Empid  Ename
1   1002   Anil
3   1004   Hema
4   1005  Laxmi
```

In [174…

```
# print(sorted(df['DOJ'], reverse=True))
df1 = df.sort_values('Salary', ascending=True)
# df.info()
# df1 = df
# df1.info()
print(df1)
```

```
   Empid   Ename    Salary         DOJ
2   1003  Gaurav      0.00    3/3/2002
0   1001  Ganesh   1000.00  10/10/2000
5   1006   Anant   9999.99    9/9/1999
4   1005   Laxmi  12000.75   10/8/2000
3   1004    Hema  16500.50   9/10/2000
1   1002    Anil  23000.50   2/20/2002
```

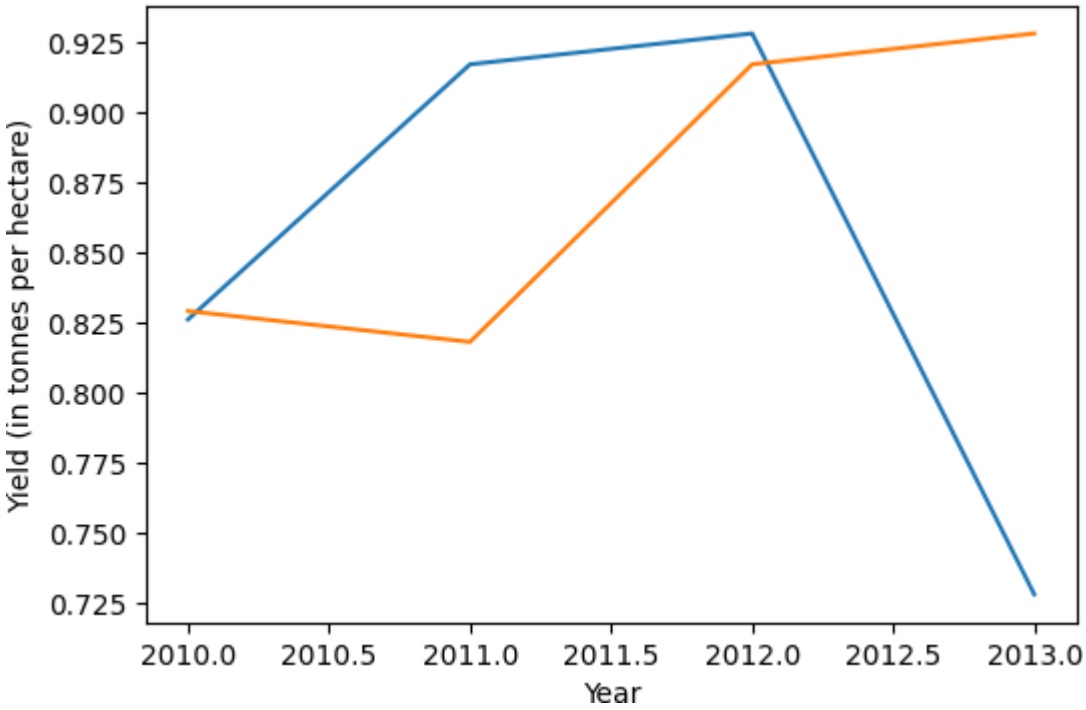
NAME: AKASH M P

ROLL NUMBER: 20221LCS0021

SECTION: 5CSE-7

In [1]:
```
import matplotlib.pyplot as plt

yield_apples = [0.826, 0.917, 0.928, 0.728]
years = [2010, 2011, 2012, 2013]

plt.figure(figsize=(6, 4))
plt.plot(years, yield_apples)
plt.xlabel('Year')
plt.ylabel('Yield (in tonnes per hectare)')

yield_oranges = [0.829, 0.818, 0.917, 0.928]
plt.plot(years, yield_oranges)
```

Out[1]:
```
[<matplotlib.lines.Line2D at 0x1ef490fb110>]
```



ls - line style

marker

color = c

alpha = transparency

ms = marker size

mec = marker edge color

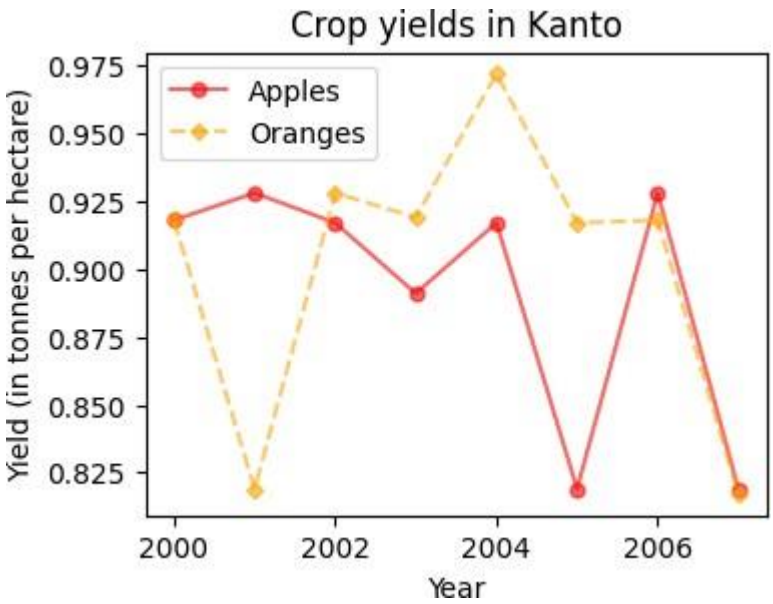
mew = marker edge width

lw = line width

In [2]:
```
years = range(2000, 2008)
apples = [0.918, 0.928, 0.917, 0.891, 0.917, 0.819, 0.928, 0.819]
oranges = [0.918, 0.819, 0.928, 0.919, 0.972, 0.917, 0.918, 0.817]
```

```
plt.figure(figsize=(4, 3))
plt.plot(years, apples, marker='o', c='red', lw=1.5, ms=5, alpha=0.6)
plt.plot(years, oranges, marker='D', c='orange', ls='--', ms=4, alpha=0.6)
plt.xlabel('Year')
plt.ylabel('Yield (in tonnes per hectare)')
plt.title('Crop yields in Kanto')
plt.legend(['Apples', 'Oranges'])
```

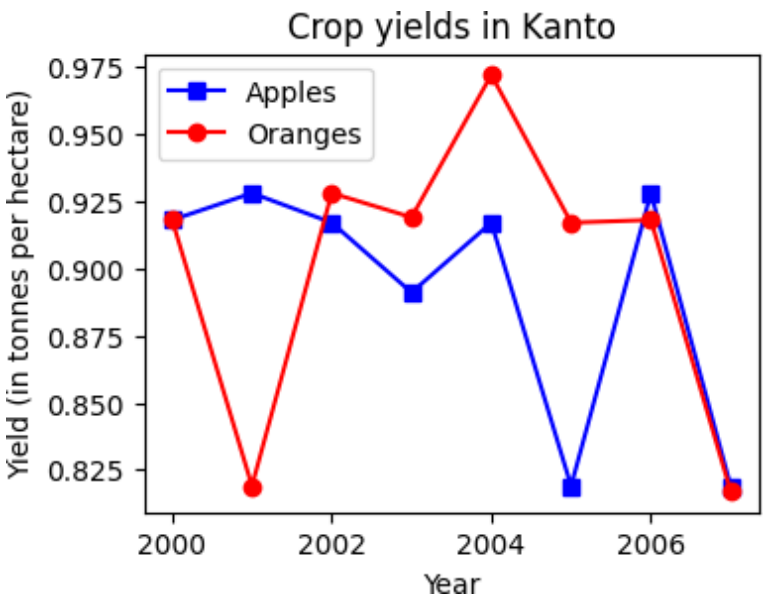
Out[2]: <matplotlib.legend.Legend at 0x1ef4b15a750>



marker, line style, color

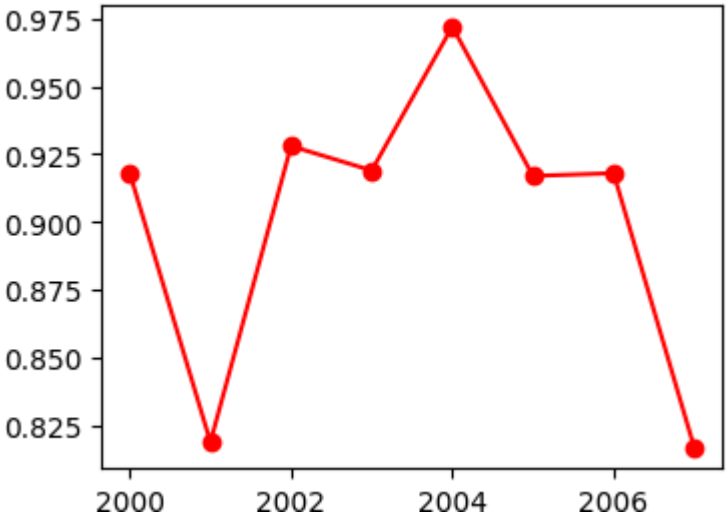
In [3]:
```
plt.figure(figsize=(4, 3))
plt.plot(years, apples, 's-b')
plt.plot(years, oranges, 'o-r')
plt.xlabel('Year')
plt.ylabel('Yield (in tonnes per hectare)')
plt.title('Crop yields in Kanto')
plt.legend(['Apples', 'Oranges'])
```

Out[3]: <matplotlib.legend.Legend at 0x1ef4b1e3980>



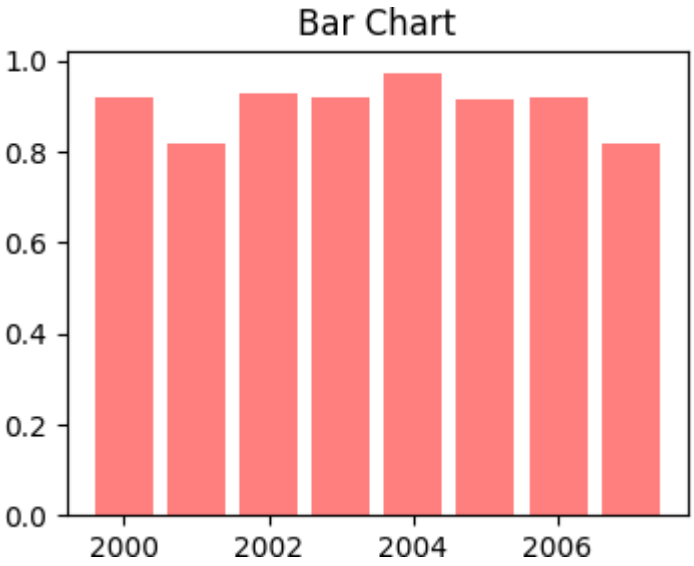
In [4]:
```
plt.figure(figsize=(4, 3))
plt.plot(years, oranges, 'o-r')
```

Out[4]:
```
[<matplotlib.lines.Line2D at 0x1ef4b2a10a0>]
```



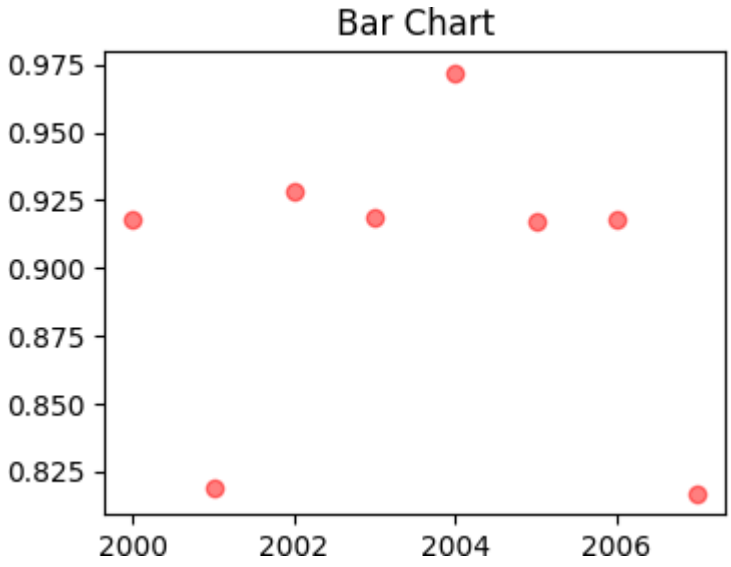
In [5]:
```
plt.figure(figsize=(4, 3))
plt.bar(years, oranges, color='r', alpha=0.5)
plt.title('Bar Chart')
```

Out[5]:
```
Text(0.5, 1.0, 'Bar Chart')
```



In [6]:
```
plt.figure(figsize=(4, 3))
plt.scatter(years, oranges, color='r', alpha=0.5)
plt.title('Bar Chart')
```

Out[6]:
```
Text(0.5, 1.0, 'Bar Chart')
```



PIE CHART

In [7]:
```
# data
x = [1, 2, 3, 4]

# explode
e = (0.1, 0, 0, 0)

lightened_colors = plt.cm.Set3.colors  # You can use any colormap you prefer
lightened_colors = [(r, g, b, 0.9) for r, g, b in lightened_colors]

plt.figure(figsize=(5, 4))
plt.pie(x, explode=e, colors=lightened_colors)
plt.title('Pie Chart')
```

Out[7]: Text(0.5, 1.0, 'Pie Chart')

LABSHEET NO: 3	REGRESSION MODELS: SIMPLE LINEAR REGRESSION	PAGE NO :
DATE:		

SIMPLE LINEAR REGRESSION:

- It aims to find a linear relationship to describe the **correlation** between an **independent** and possibly **dependent variable**. The regression line can be used to predict or estimate missing values, this is known as *interpolation.*
- "Linear regression predicts the relationship between two variables by assuming a linear connection between the independent and dependent variables. It seeks the optimal line that minimizes the sum of squared differences between predicted and actual values.
- The goal of the linear regression algorithm is to get the best values for B0 and B1 to find the best fit line.
- The Simple Linear Regression model can be represented using the below equation:

  $y= a0+a1x+ \varepsilon$

a0= It is the intercept of the Regression line (can be obtained putting x=0)

a1= It is the slope of the regression line, which is either increasing or decreasing.

ε = The error term. (For a good model it will be negligible)

TWO OBJECTIVES:

- **Model the relationship between the two variables.** Such as the relationship between Income and expenditure, experience and Salary, etc.
- **Forecasting new observations.** Such as Weather forecasting according to temperature, Revenue of a company according to the investments in a year, etc

STEPS TO EXECUTE SIMPLE LINEAR REGRESSION

1. Import the necessary Libraries.
2. Convert Dataset to Data frame.
3. View Descriptive statistics.
4. Detect Outlier Analysis.
5. Declare Feature variable and Target Variable.
6. Plot scatter Plot between X and Y
7. Print the dimensions of X and Y
8. Reshape and Print the Dimensions of X and Y after reshaping
9. Split X and Y into training and test data sets
10. Fit the Linear Model
11. Visualizing the Training set results
12. Visualizing the Test set results
13. Compute Model Slope and Intercept

NAME: AKASH M P

ROLL NUMBER: 20221LCS0021

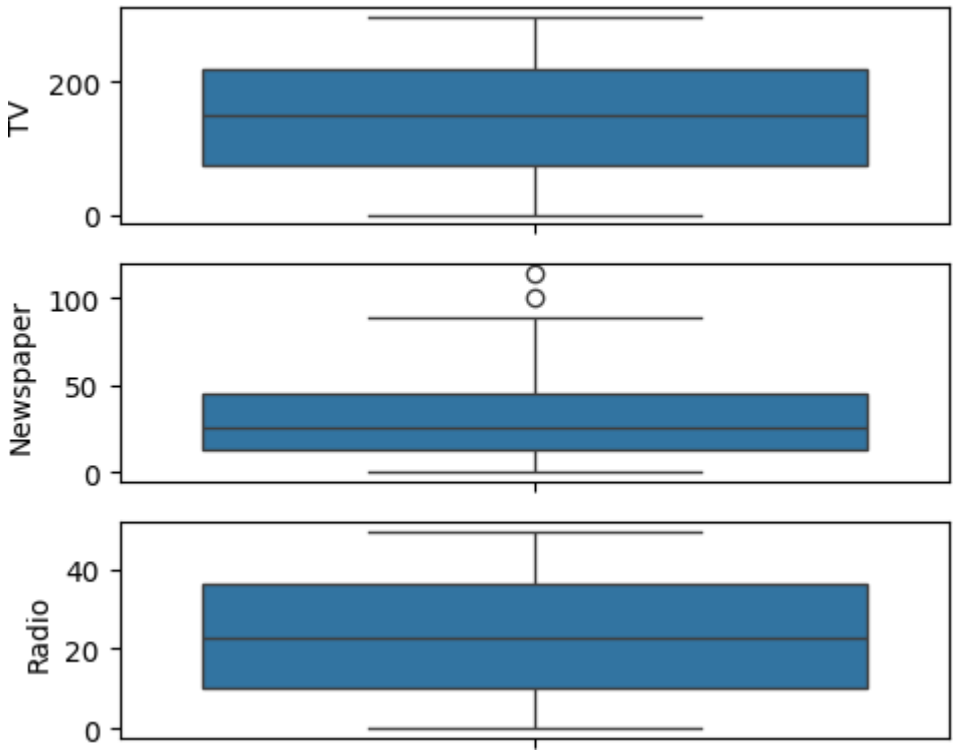
SECTION: 5CSE-7

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
df = pd.read_csv('advertising.csv')
print(df.head())
df.info()
print(df.shape)
print(df.describe())    # basic statistics of the file
```

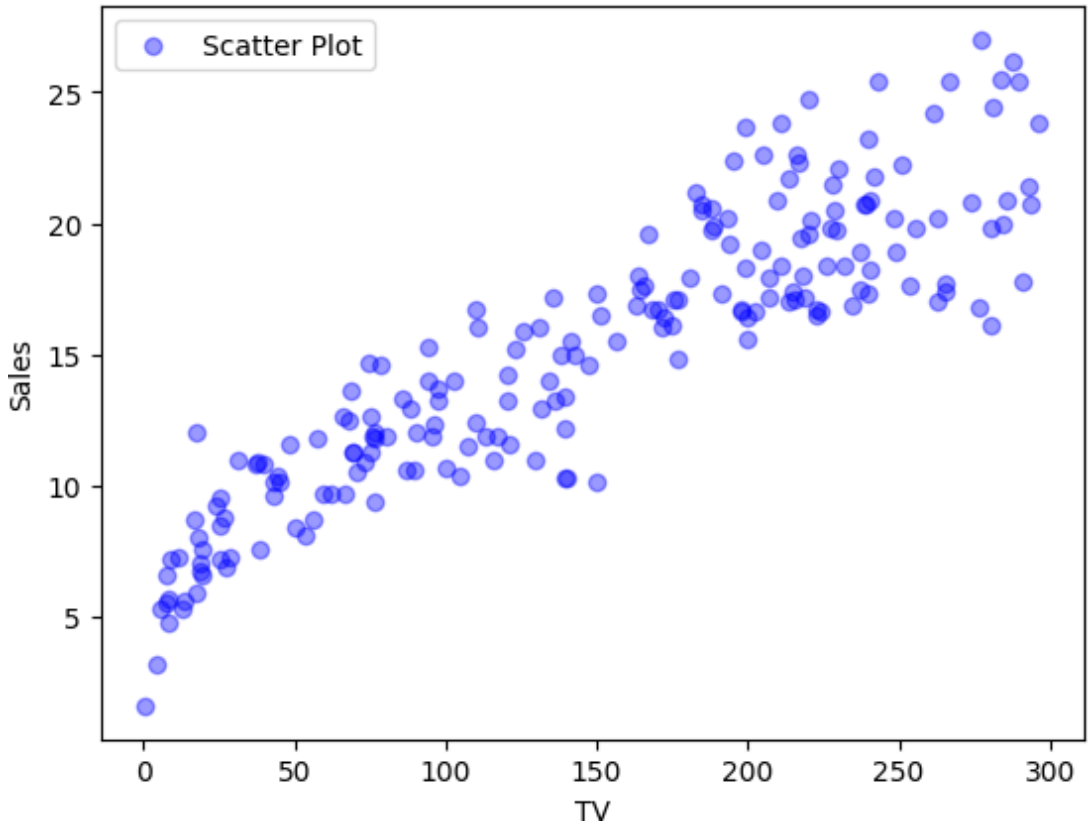
```
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
(200, 4)
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
```

In [3]:
```
# Outlier analysis
fig, axs = plt.subplots(3, figsize = (5, 4))
plt1 = sns.boxplot(df['TV'], ax = axs[0])
plt2 = sns.boxplot(df['Newspaper'], ax = axs[1])
plt3 = sns.boxplot(df['Radio'], ax = axs[2])
plt.tight_layout()
```



In [4]:
```
X = df['TV']
Y = df['Sales']
```

In [5]:
```
plt.scatter(X, Y, color = 'blue', alpha=0.4, label = 'Scatter Plot')
plt.xlabel('TV')
plt.ylabel('Sales')
plt.legend()
plt.show()
```



In [6]:
```
x = np.array(X)
y = np.array(Y)

print(X.shape)
print(Y.shape)
```

```
(200,)
(200,)
```

In [7]:
```
x = x.reshape(-1, 1)
y = y.reshape(-1, 1)

print(x.shape)
print(y.shape)
```

```
(200, 1)
(200, 1)
```

In [8]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.30, rand
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(140, 1)
(60, 1)
(140, 1)
(60, 1)
```

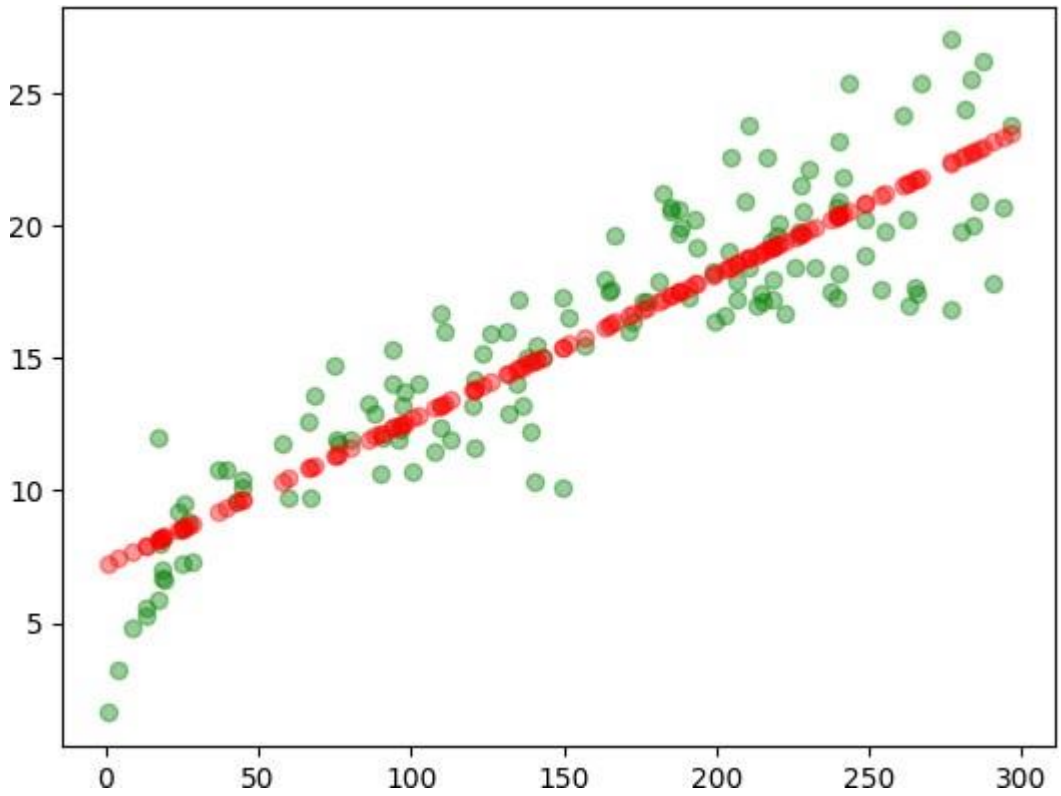
In [9]:
```
from sklearn.linear_model import LinearRegression
lm = LinearRegression()

# lm is the model
# Train model using data sets
lm.fit(X_train, y_train)
```

Out[9]:
```
▾ LinearRegression
LinearRegression()
```

In [10]:
```
y_pred = lm.predict(X_test)
```

In [11]:
```
plt.scatter(X_train, y_train, color = 'green', alpha=0.4)
plt.scatter(X_train, lm.predict(X_train), color = 'red', alpha=0.4)
```

Out[11]:
```
<matplotlib.collections.PathCollection at 0x1d670ccda00>
```



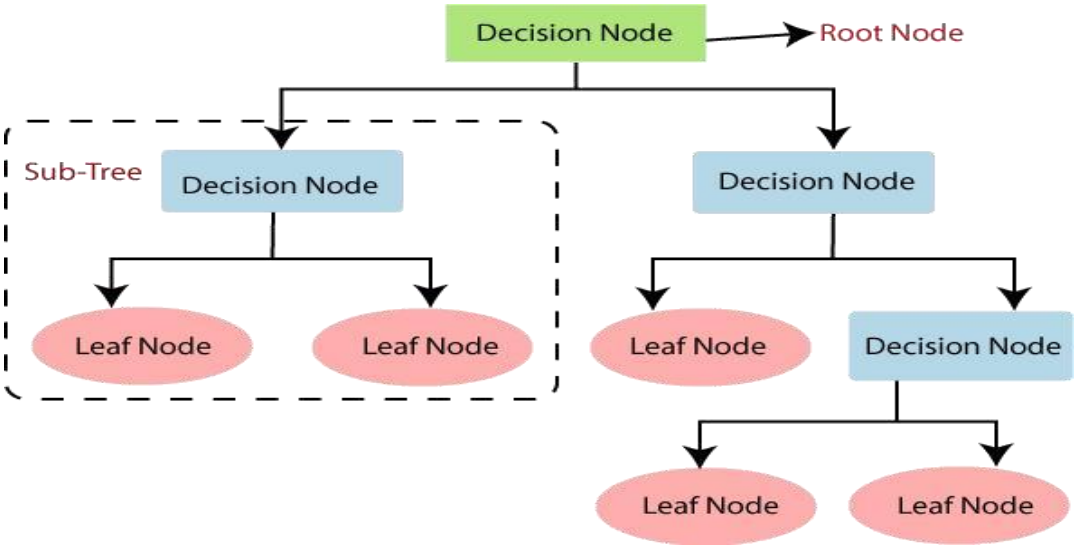
In [12]:
```
slope = lm.coef_
intercept = lm.intercept_
print('Estimated model slope: ', slope)
print('Estimated model intercept: ', intercept)
```

```
Estimated model slope:  [[0.05483488]]
Estimated model intercept:  [7.20655455]
```

LABSHEET NO: 4	DECISION TREE CLASSIFIERS USING GINI INDEX	PAGE NO :
DATE:		

DECISION TREE CLASSIFIER:

- Decision Tree is a supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.
- In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the CART algorithm, which stands for Classification and Regression Tree algorithm



GINI INDEX:

Formula of the Gini Index is as follows:

Gini=1−n∑i=1(pi)2
where, 'pi' is the probability of an object being classified to a particular class.

How does Decision Tree Algorithm works?

Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

Step-2: Find the best attribute in the dataset using Attribute Selection Measure (ASM).

Step-3: Divide the S into subsets that contains possible values for the best attributes. **Step-**

4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

STEPS TO EXECUTE DECISION TREE USING GINI INDEX

1. Load the German Credit card dataset
2. Create a Pandas Frame for this file and explore its content
3. Print the first five records and first 7 columns
4. Print the first five records and remaining columns
5. Few of the columns are categorical and are inferred as objects. Ex: checkin_acc. Print all unique values of this column
6. Encode all categorical features using one-hot encoding. A feature with n values is encoded using (n-1) values, retaining the first one (drop_first = True)
7. Make independent features of the encoded frame as X and column 'status' as dependent feature.
8. Divide data into 70% training and 30% as testing.
9. Train a decision tree model using Gini Index and depth of 3
10. Make predictions on test/validation data
11. Print the confusion matrix, accuracy and AUC score of this model on test set
12. Visualize the tree using graphviz and pydotplus libraries

NAME: AKASH M P

ROLL NUMBER: 20221LCS0021

SECTION: 5CSE-7

DECISION TREE CLASSIFIER USING GINI INDEX AS IMPURITY MEASURE

In [1]:
```
import pandas as pd
df = pd.read_csv("German_Credit_Card_Dataset.csv")
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   checkin_acc        1000 non-null   object
 1   duration           1000 non-null   int64
 2   credit_history     1000 non-null   object
 3   amount             1000 non-null   int64
 4   savings_acc        1000 non-null   object
 5   present_emp_since  1000 non-null   object
 6   inst_rate          1000 non-null   int64
 7   personal_status    1000 non-null   object
 8   residing_since     1000 non-null   int64
 9   age                1000 non-null   int64
 10  inst_plans         1000 non-null   object
 11  num_credits        1000 non-null   int64
 12  job                1000 non-null   object
 13  status             1000 non-null   int64
dtypes: int64(7), object(7)
memory usage: 109.5+ KB
```

Out[1]:

	checkin_acc	duration	credit_history	amount	savings_acc	present_emp_since	inst_r
0	A11	6	A34	1169	A65	A75	
1	A12	48	A32	5951	A61	A73	
2	A14	12	A34	2096	A61	A74	
3	A11	42	A32	7882	A61	A74	
4	A11	24	A33	4870	A61	A73	

In [2]:
```
df.iloc[0:5,0:7]
```

Out[2]:

	checkin_acc	duration	credit_history	amount	savings_acc	present_emp_since	inst_r
0	A11	6	A34	1169	A65	A75	
1	A12	48	A32	5951	A61	A73	
2	A14	12	A34	2096	A61	A74	
3	A11	42	A32	7882	A61	A74	
4	A11	24	A33	4870	A61	A73	

In [3]:
```
df.iloc[0:5, 7:15]
```

Out[3]:

	personal_status	residing_since	age	inst_plans	num_credits	job	status
0	A93	4	67	A143	2	A173	0
1	A92	2	22	A143	1	A173	1
2	A93	3	49	A143	1	A172	0
3	A93	4	45	A143	1	A173	0
4	A93	4	53	A143	2	A173	1

In [4]:
```
df['checkin_acc'].unique()
```

Out[4]:
```
array(['A11', 'A12', 'A14', 'A13'], dtype=object)
```

Encode all categorical features using one-hot encoding. A feature with n values is encoded using (n-1) values, retaining the first one (drop_first = True)

In [5]:
```
X_features = list(df.columns)
X_features.remove('status')
encoded_df  =  pd.get_dummies(df[X_features],drop_first=True)
print(list(encoded_df.columns))
```

```
['duration', 'amount', 'inst_rate', 'residing_since', 'age', 'num_credits', 'chec
kin_acc_A12', 'checkin_acc_A13', 'checkin_acc_A14', 'credit_history_A31', 'credit
_history_A32', 'credit_history_A33', 'credit_history_A34', 'savings_acc_A62', 'sa
vings_acc_A63', 'savings_acc_A64', 'savings_acc_A65', 'present_emp_since_A72', 'p
resent_emp_since_A73', 'present_emp_since_A74', 'present_emp_since_A75', 'persona
l_status_A92', 'personal_status_A93', 'personal_status_A94', 'inst_plans_A142',
'inst_plans_A143', 'job_A172', 'job_A173', 'job_A174']
```

Make independent features of the encoded frame as X and column 'status' as dependent feature.

In [6]:
```
X = encoded_df
Y = df['status']
```

Divide data into 70% training and 30% as testing.

In [7]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,Y,test_size = 0.3, random_
```

Train a decision tree model using Gini INdex and depth of 3

In [8]:
```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(criterion = 'gini', max_depth = 3)
clf.fit(X_train, y_train)
```

Out[8]:
```
▾ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=3)
```

Make predictions on test/validation data

In [9]:
```
pred_y = clf.predict(X_test)
```

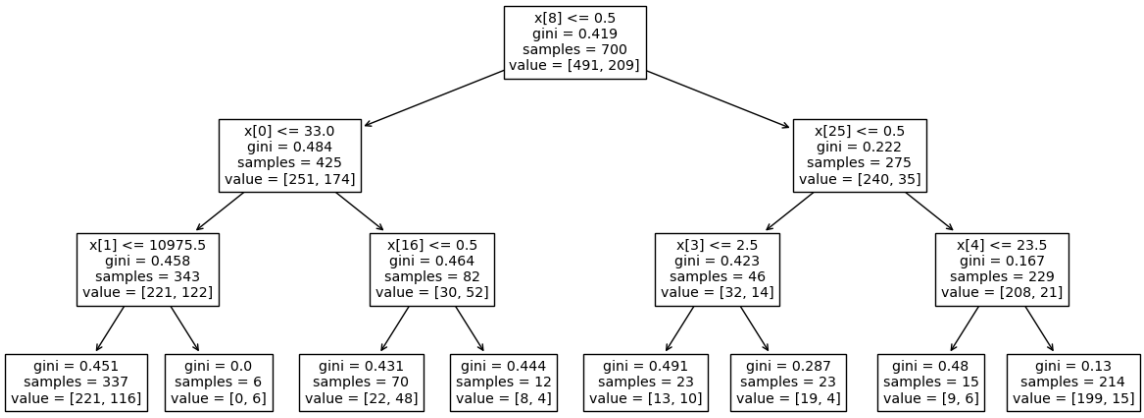
Print the confusion matrix, accuracy and AUC score of this model on test set

In [10]:
```
from sklearn import metrics
print("Confusion Matrix is\n",metrics.confusion_matrix(pred_y, y_test))
print("Accuracy is", metrics.accuracy_score(pred_y,y_test))
print("AUC Score is", metrics.roc_auc_score(pred_y, y_test))
```

```
Confusion Matrix is
 [[198  71]
 [ 11  20]]
Accuracy is 0.7266666666666667
AUC Score is 0.6906103849382419
```

Visualize the tree using grapghviz and pydotplus libraries

In [11]:
```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
plt.figure(figsize=(15, 6))
plot_tree(clf, feature_names=None, class_names=None)
plt.show()
```



LABSHEET NO: 5	DEVELOPING A GRADIENT DESCENT ALGORITHM FOR LINEAR REGRESSION	PAGE NO :
DATE:		

GRADIENT DESCENT ALGORITHM:

- Gradient Descent is an algorithm that finds the best-fit line for a given training dataset in a smaller number of iterations.
- A gradient simply measures the change in all weights with regard to the change in error. You can also think of a gradient as the slope of a function. The higher the gradient, the steeper the slope and the faster a model can learn. But if the slope is zero, the model stops learning.
- A Linear Regression model converging to optimum solution using Gradient Descent.
- Gradient Descent is used for multiple linear regression as an iterative algorithm use in loss function to find the global minima. The loss can be any differential loss function.
- **Real time applications:** Calculate the gradients of slopes in and around school, like staircases and ramps.

Types of Cost Function

Cost functions can be of various types depending on the problem.

- *Regression Cost Function:* Regression models are used to make a prediction for the continuous variables such as the price of houses, weather prediction, loan predictions, etc. When a cost function is used with Regression, it is known as the "Regression Cost Function."

Error= Actual Output-Predicted output

Three commonly used Regression cost functions

1. **Means Error**
2. **Mean Squared Error (MSE)**
3. **Mean Absolute Error (MAE)**

- ***Binary Classification cost Functions:*** Classification models are used to make predictions of categorical variables, such as predictions for 0 or 1, Cat or dog, etc. The cost function used in the classification problem is known as the Classification cost function.

- ***Multi-class Classification Cost Function:*** A multi-class classification cost function is used in the classification problems for which instances are allocated to one of more than two classes.

STEPS:

1. Let m = 0 and c = 0. Let L be our learning rate. It could be a small value like 0.01 for good accuracy
2. Calculate the partial derivative of the Cost function with respect to m. Let partial derivative of the Cost function with respect to m be D_m (With little change in m how much Cost function changes).
3. Now update the current values of m and c using the following equation
4. We will repeat this process until our Cost function is very small (ideally 0).
5. **Gradient Descent Algorithm** gives optimum values of m and c of the linear regression equation.

STEPS TO EXECUTE GRADIENT DESCENT ALGORITHM:

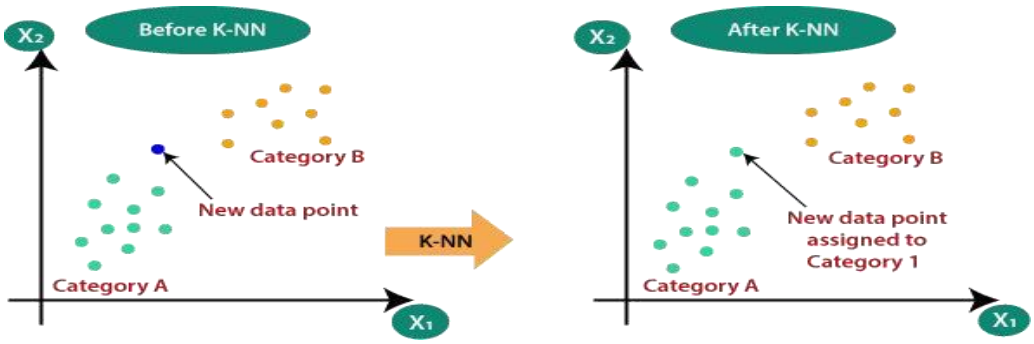
1. Load the Data set
2. Grab the relevant data, scale the predictor variable, and add a column of 1s for the gradient descent.
3. Plot the Cost Function
4. Generate the Animation Data
5. Display the Animation

K NEAREST NEIGHBOR CLASSIFIER:

- K-Nearest Neighbor is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Why do we need a K-NN Algorithm?

- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



STEPS TO EXECUTE K NEAREST NEIGHBOR ALGORITHM USING GRID SEARCH CV:

1. Upload the Dataset and import the Libraries.
2. Load the Dataset into frame.
3. Print the description, dimensions and first five records of the frame
4. Check whether the data has any missing value in any column.
5. Check whether the data has balanced class distribution. Class target = 0 indicates "Heart Disease" and target = 1 indicates "No Heart Disease".
6. Create input features X, target Y, classifier object, train-test-split using 80-20% split
7. Train model
8. Validate model on test set
9. Print Classification Report on test data
10. Print AUC score on test data
11. Hyper parameter tuning using Grid Search CV. Set the parameters a) leaf-size= 1 to 15, b)n_neighbors = 1 to 10 and c) distance metric, p = 1, 2. When p =1 its Manhattan and p = 2 its Euclidean distance. Grid Search CV uses CV to search for the optimal values of the hyper parameters. It accepts the hyper parameters as a dictionary.
12. Train a new kNN model using Grid Search CV.
13. Print the best values of the hyper parameter
14. Validate the model on test data
15. Print classification report and AUC score of the model on test data

NAME: AKASH M P

ROLL NUMBER: 20221LCS0021

SECTION: 5CSE-7

In [1]:
```
import pandas as pd
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
df = pd.read_csv('heart.csv')
print(df.info())
print(df.shape)
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
None
(303, 14)
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   63    1   3       145   233    1        0      150      0      2.3      0
1   37    1   2       130   250    0        1      187      0      3.5      0
2   41    0   1       130   204    0        0      172      0      1.4      2
3   56    1   1       120   236    0        1      178      0      0.8      2
4   57    0   0       120   354    0        1      163      1      0.6      2

   ca  thal  target
0   0     1       1
1   0     2       1
2   0     2       1
3   0     2       1
4   0     2       1
```

In [4]:
```
x = df.drop(columns=['target'])
y = df['target']
knn = KNeighborsClassifier()
```

```
X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.2, random_
knn.fit(X_train, Y_train)
```

Out[4]:

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

In [5]:

```
y_pred = knn.predict(X_test)
print(y_pred)
```

```
[0 1 1 1 0 1 1 0 0 1 0 0 1 1 1 1 1 1 1 0 0 0 0 1 1 1 0 1 0 1 0 0 0 1 0 0 1
 1 1 1 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 0 1 0 1 0]
```

In [7]:

```
print(classification_report(Y_test,  y_pred))
```

```
              precision    recall  f1-score   support

           0       0.69      0.62      0.65        29
           1       0.69      0.75      0.72        32

    accuracy                           0.69        61
   macro avg       0.69      0.69      0.69        61
weighted avg       0.69      0.69      0.69        61
```

In [8]:

```
roc_auc_score(Y_test,  y_pred)
```

Out[8]: 0.6853448275862069

Hyperparameter tuning using GridSearchCV. Set the params:

a. leaf_size = 1 to 15

b. n_neighbors = 1 to 10

c. distance metric, p = 1, 2 When p = 1, it's Manhattan, p = 2, it's Euclidean distance

GridSearchCV uses CV to search for the optimal values of hyperparams. It accepts the hyperparams as dictionary.

In [9]:

```
leaf_size = list(range(1, 15))
n_neighbors = list(range(1, 10))
p = [1, 2]
hyperparams = dict(leaf_size = leaf_size, n_neighbors = n_neighbors, p = p)
print(hyperparams)
```

```
{'leaf_size': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14], 'n_neighbors': [1,
2, 3, 4, 5, 6, 7, 8, 9], 'p': [1, 2]}
```

In [10]:

```
knn_2 = KNeighborsClassifier()
clf = GridSearchCV(knn_2, hyperparams, cv = 10, scoring='roc_auc')
best_model = clf.fit(x, y)
```

In [11]:

```
print("Best leaf size:", best_model.best_estimator_.get_params()['leaf_size'])
print("Best p:", best_model.best_estimator_.get_params()['p'])
print("Best n_neighbors:", best_model.best_estimator_.get_params()['n_neighbors'
print("Best score:", best_model.best_score_)
```

```
Best leaf size: 1
Best p: 1
Best n_neighbors: 7
Best score: 0.7470143826761475
```

In [12]:
```
new_y_pred = best_model.predict(X_test)
```

In [13]:
```
print(classification_report(Y_test, new_y_pred))
roc_auc_score(Y_test, new_y_pred)
```

```
              precision    recall  f1-score   support

           0       0.76      0.86      0.81        29
           1       0.86      0.75      0.80        32

    accuracy                           0.80        61
   macro avg       0.81      0.81      0.80        61
weighted avg       0.81      0.80      0.80        61
```

Out[13]: 0.8060344827586207

LABSHEET NO: 7	ENSEMBLE MODELS	PAGE NO :
DATE:		

ENSEMBLE LEARNING

- Ensemble means combining multiple models.
- Predict class label of previously unseen records by aggregating predictions made by multiple classifiers
- Improves the classification accuracy
- Predicted output of the base classifiers is combined by majority voting
- Build different experts and let them vote.
- Build many models and combine them.
- Only through averaging do we get at the truth!
- It's too hard (*impossible*?) to build a single model that works best
- Two types of approaches:
    - Models that don't use randomness
    - Models that incorporate randomness

STEPS TO EXECUTE ENSEMBLE LEARNING:

1. Implement the Bagging based Ensemble Model using CART (Classification and Regression Trees) as base learners. No. of base learners = 100. Use cross validation as the model estimation method.
2. Implement the Ada Boost Ensemble model for classification using 10 base learners and cross validation
3. Implement the Bagging based Ensemble Model using k-Nearest Neighbor Classifier as base learners. No. of base learners = 100. Use cross validation as the model estimation method.
4. Find best accuracy by implementing in different learning methods.

NAME: AKASH M P
ROLL NUMBER: 20221LCS0021
SECTION: 5CSE-7

Implement the bagging based Ensemble Model using CART (Classification and Regression Trees) as base learners. No. of base learners = 100. Use cross validation as the model estimation method.

In [1]:
```
import pandas as pd
from sklearn import model_selection
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
df = pd.read_csv(url, names=names)
df
```

Out[1]:

	preg	plas	pres	skin	test	mass	pedi	age	class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

In [2]:
```
import numpy as np
import warnings
warnings.filterwarnings("ignore")
array = df.values
X = array[:, 0:8]
Y = array[:, 8]
kfold = model_selection.KFold(n_splits=10, random_state=None)
clf = DecisionTreeClassifier()
num_trees = 100
model = BaggingClassifier(base_estimator=clf, n_estimators=num_trees, random_sta
results = model_selection.cross_val_score(model, X, Y, cv=kfold)
average_accuracy = np.mean(results)
print("Average Accuracy:", average_accuracy)
```

```
Average Accuracy: 0.7720437457279563
```

Implement the AdaBoost Ensemble model for classification using 10 base learners and cross validation

In [3]:
```
from sklearn.ensemble import AdaBoostClassifier
```

In [4]:
```
num_trees = 10
model = AdaBoostClassifier(n_estimators=num_trees, random_state=7)
results = model_selection.cross_val_score(model, X, Y, cv=kfold)
average_accuracy = np.mean(results)
print("Average Accuracy:", average_accuracy)
```

```
Average Accuracy: 0.7525974025974026
```

Implement the Bagging based Ensemble Model using KNN Classifier as base learners. No. of base learners = 100. Use CV as the model estimation method.

In [5]:
```
from sklearn.neighbors import KNeighborsClassifier
```

In [6]:
```
clf = KNeighborsClassifier()
num_trees = 100
model = BaggingClassifier(base_estimator=clf, n_estimators=num_trees, random_sta
results = model_selection.cross_val_score(model, X, Y, cv=kfold)
average_accuracy = np.mean(results)
print("Average Accuracy:", average_accuracy)
```

```
Average Accuracy: 0.7356459330143541
```

Implement the Bagging based Ensemble model for Gaussian Naive Bayes using 100 base learners and cross validation

In [7]:
```
from sklearn.naive_bayes import GaussianNB
```

In [8]:
```
clf = GaussianNB()
model = BaggingClassifier(base_estimator=clf, n_estimators=num_trees, random_sta
results = model_selection.cross_val_score(model, X, Y, cv=kfold)
average_accuracy = np.mean(results)
print("Average Accuracy:", average_accuracy)
```

```
Average Accuracy: 0.753879015721121
```

LABSHEET NO: 8	K MEANS CLUSTERING	PAGE NO :
DATE:		

K MEANS CLUSTERING

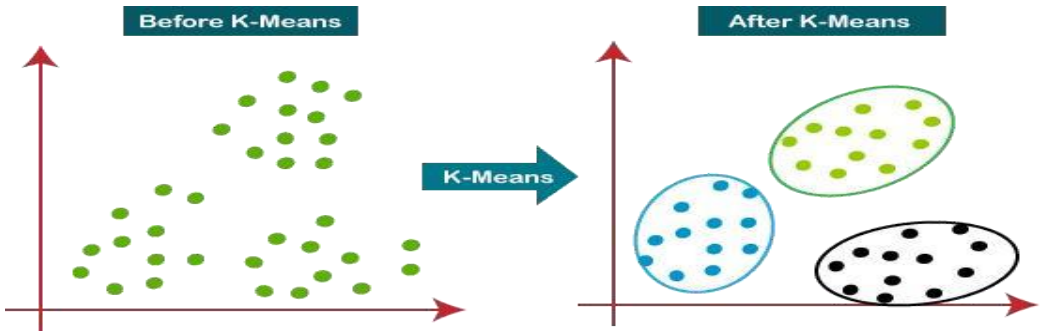
- K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science

What is K-Means Algorithm?

- K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.

The k-means clustering algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.
- Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster



STEPS TO EXECUTE K - MEANS CLUSTERING:

1. Upload the Data set.
2. Create a data frame and visualize the natural groupings in the dataset
3. The groupings are mostly segmented using income, since it has a huge range. Scale of age is 0 to 60 and income is from 0 to 50000. Hence Euclidean distance will always be dominated by income and not age. Hence all features need to be normalized to a uniform scale before clustering.
4. Plotting customers with their segments
5. Print the cluster centers using the original data frame. Cluster centers explain the characteristics of the cluster and helps us to interpret the clusters. Print the cluster centers to understand the average age and income of each cluster.
6. So Cluster 0 has a mean age of 46 and income of 43K. High age and medium income. Cluster 1 has a mean age of 39 and income of 18K. Mid age and low income. Cluster 2 has a mean age of 31 and income of 54K. Low age and high income. The actual age and income of a customer within a cluster will vary from the cluster centers and is called the cluster variance. This is given by WCSS - within cluster sum of squares.
7. Find the optimum number of clusters that may exist using Elbow Method. Try with number of clusters from 1 to 10. In each case print the total variance using "inertia" parameter of the clusters.
8. The figure indicates the elbow point is 3, this means there might exist three clusters in the data set.

NAME: AKASH M P
ROLL NUMBER: 20221LCS0021
SECTION: 5CSE-7

Implement K-means clustering using 'Income Data.csv'

In [1]:
```
import pandas as pd
df = pd.read_csv('Income Data.csv')
df
```

Out[1]:

	income	age
0	41100.0	48.75
1	54100.0	28.10
2	47800.0	46.75
3	19100.0	40.25
4	18200.0	35.80
...	...	...
295	38300.0	46.10
296	27400.0	44.15
297	54200.0	28.25
298	41800.0	44.85
299	38600.0	39.35

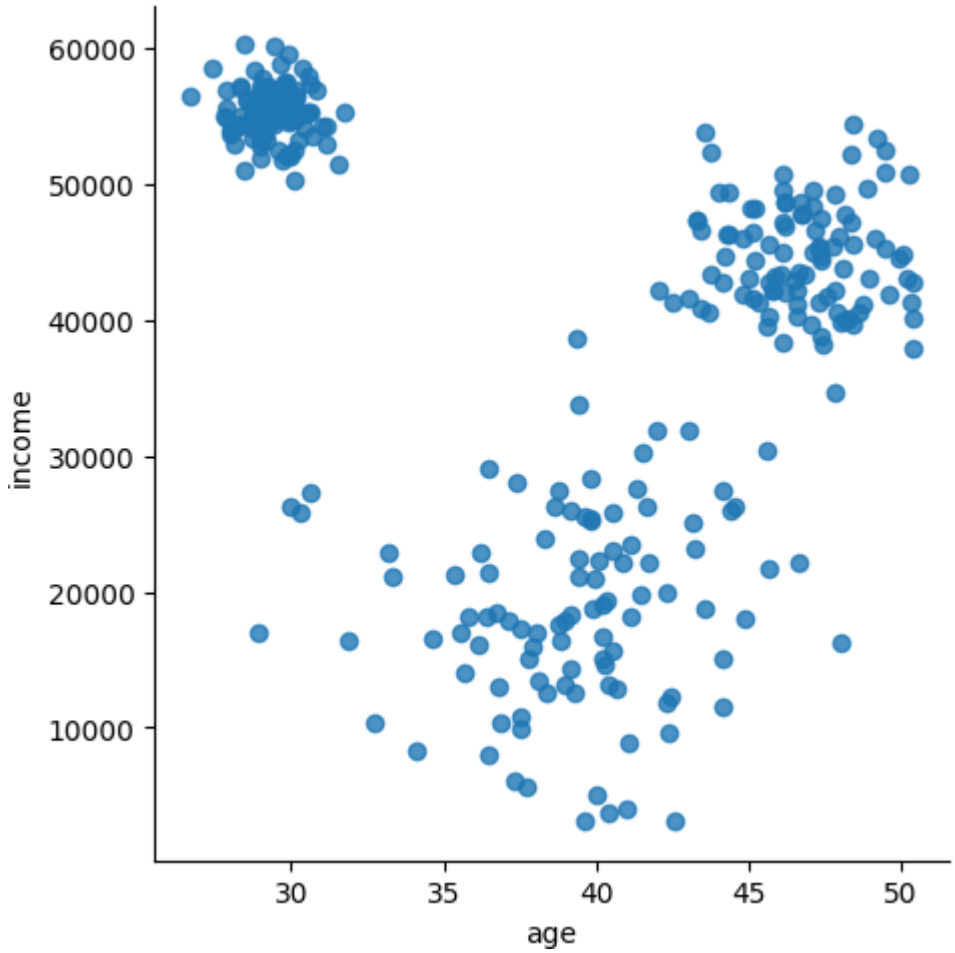
300 rows × 2 columns

Create a dataframe and visualize the natural grouping in the dataset

In [2]:
```
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

In [3]:
```
sns.lmplot(data=df, x='age', y='income', fit_reg=False)
```

Out[3]:
```
<seaborn.axisgrid.FacetGrid at 0x20ee3e57170>
```



The above grouping are mostly segmented using income, since it has a huge range. Scale of 0 to 60 and income is from 0 to 50000. Hence, Euclidean distance will always be dominated by income and not age. Hence, all features need to be normalized to a uniform scale before clustering

In [4]:
```
from sklearn.preprocessing import StandardScaler
```

In [5]:
```
scaler = StandardScaler()
scaled_df = scaler.fit_transform(df[["age", "income"]])
scaled_df[0:5]
```

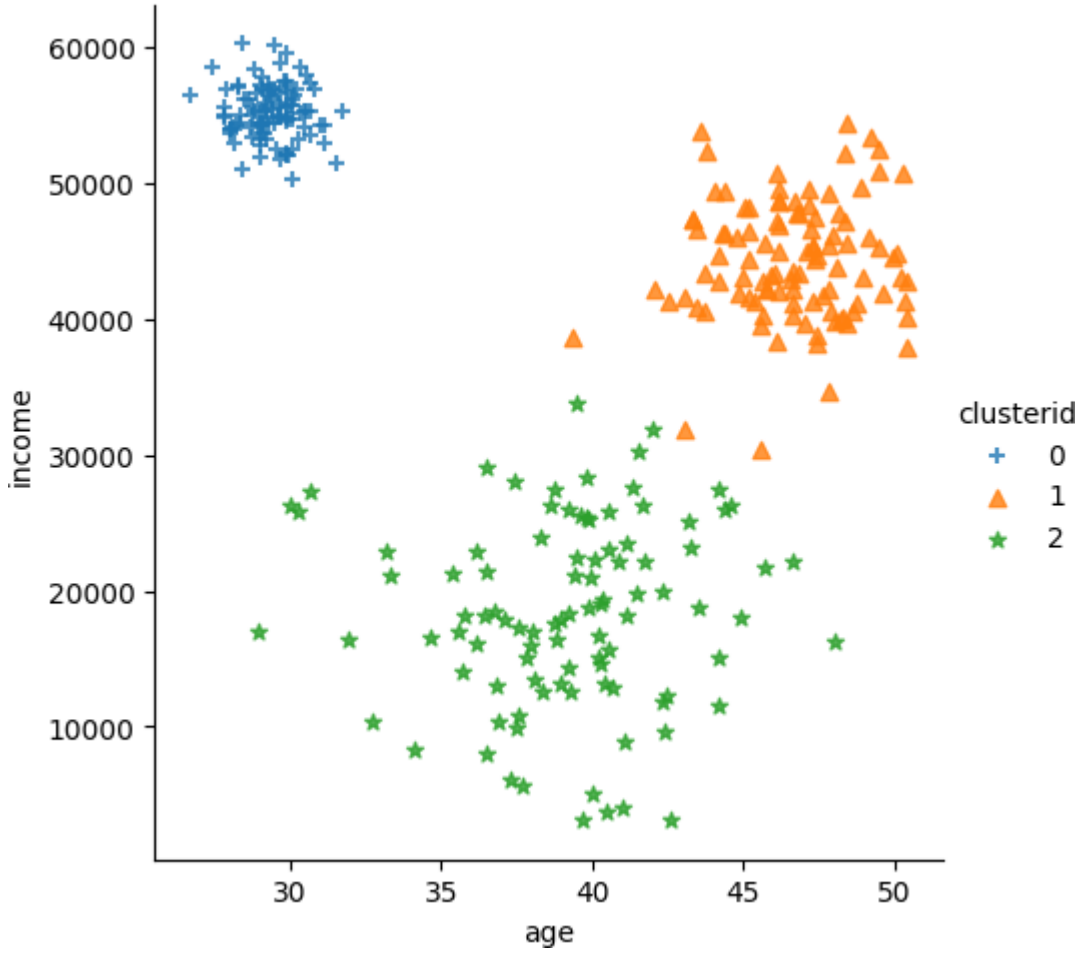
Out[5]:
```
array([[ 1.3701637 ,  0.09718548],
       [-1.3791283 ,  0.90602749],
       [ 1.10388844,  0.51405021],
       [ 0.23849387, -1.27162408],
       [-0.35396857, -1.32762083]])
```

Plotting customers with the segments

In [6]:
```
from sklearn.cluster import KMeans
```

In [7]:
```
import warnings
warnings.filterwarnings("ignore")
clusters = KMeans(3)
clusters.fit(scaled_df)
df["clusterid"] = clusters.labels_
markers = ['+', '^', '*']
sns.lmplot(x="age", y="income", data = df, hue = "clusterid", fit_reg = False, m
```

Out[7]: <seaborn.axisgrid.FacetGrid at 0x20ee6571040>



Print the cluster centers using the original dataframe. Clutser centers explain the characteristics of the clusters and helps us to interpret the clusters. Print the cluster to understand the average age and income of each other.

In [8]:
```
clusters = KMeans(3)
clusters.fit(scaled_df)
df["new_clusterid"] = clusters.labels_
df.groupby("new_clusterid")[['age', 'income']].agg(["mean", 'std']).reset_index(
```

Out[8]:

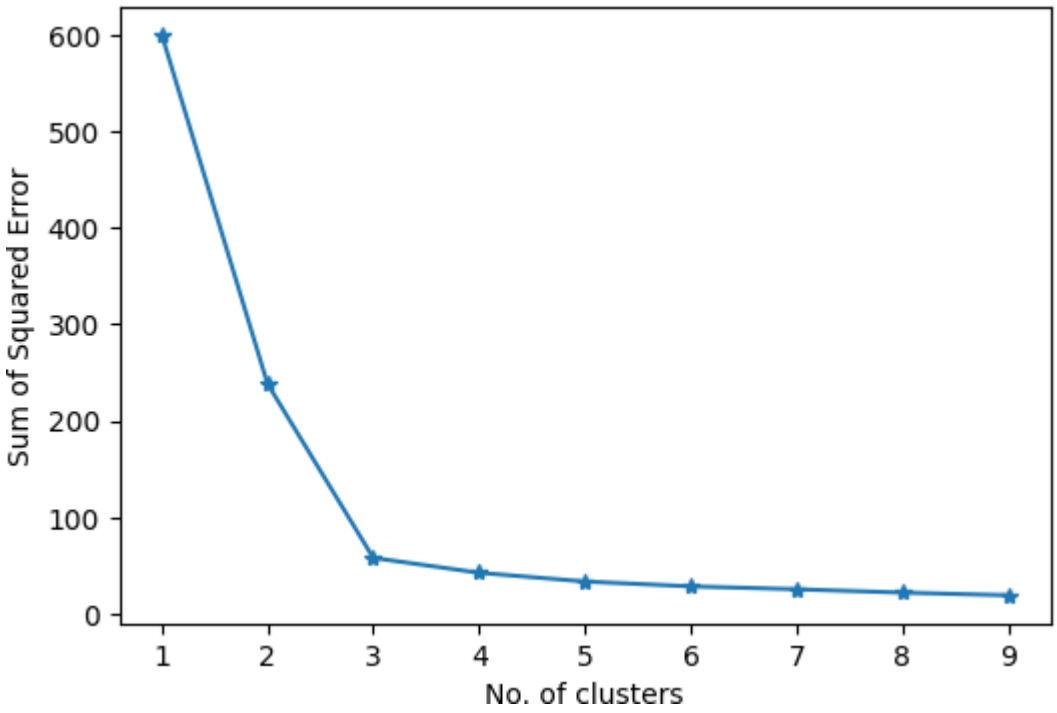
new_clusterid		age		income	
		mean	std	mean	std
0	0	39.140206	3.558665	18321.649485	6924.747691
1	1	29.384000	0.921458	55204.000000	1951.943864
2	2	46.627184	2.151559	44308.737864	4390.321503

So Cluster 0 has a mean age of 39 and income of 18K. Low age and low income. CLuster 1 has a mean age of 37 and income of 54K. Mid age and high income. CLuster 2 has a mean age of 46 and income of 43K. High age and medium income. The actual age and income of a customer within a cluster will vary from the cluster centers and is called the cluster variance. This is given by WCSS - within cluster sum of square
s. Find the optimum number of clusters that may exist using Elbow Method. Try with

number of clusters from 1 to 10. In each case print the total variance using "inertia" parameter of the clusters.ares

In [9]:
```
cluster_range = range(1,10)
cluster_errors = []
for num_clusters in cluster_range:
    clusters = KMeans(num_clusters)
    clusters.fit(scaled_df)
    cluster_errors.append(clusters.inertia_)
plt.figure(figsize = (6,4))
plt.plot(cluster_range, cluster_errors, marker = "*")
plt.xlabel("No. of clusters")
plt.ylabel("Sum of Squared Error")
```

Out[9]:
```
Text(0, 0.5, 'Sum of Squared Error')
```



The figure indicates the elbow point is 3, this means there might exist three clusters in the data set.

In [10]:
```
trial_df = pd.DataFrame({'Animal': ['Falcon', 'Falcon', 'Parrot', 'Parrot'], 'Ma
```

In [11]:
```
trial_df.groupby(['Animal']).mean()
```

Out[11]:

	Max Speed
Animal	
Falcon	375.0
Parrot	25.0

In [12]:
```
t_df = pd.DataFrame({'A': [1, 1, 2, 2], 'B': [1, 2, 3, 4], 'C': np.random.randn(
```

In [13]:
```
t_df.groupby('A').agg('min')
```

Out[13]:

	B	C
A		
1	1	0.130146
2	3	-0.284931

In [14]:
```
t_df.groupby('A').agg(['min', 'max'])
```

Out[14]:

	B		C	
	min	**max**	**min**	**max**
A				
1	1	2	0.130146	1.309332
2	3	4	-0.284931	0.733395

CONDITIONAL PROBABILITY

- Bayes Theorem provides a principled way for calculating a conditional probability.
- It is a deceptively simple calculation, although it can be used to easily calculate the conditional probability of events where intuition often fails.
- Although it is a powerful tool in the field of probability, Bayes Theorem is also widely used in the field of machine learning. Including its use in a probability framework for fitting a model to a training dataset, referred to as maximum a posteriori or MAP for short, and in developing models for classification predictive modeling problems such as the Bayes Optimal Classifier and Naive Bayes.
- Bayes Theorem is a method to determine conditional probabilities – that is, the probability of one event occurring given that another event has already occurred. Because a conditional probability includes additional conditions – in other words, more data – it can contribute to more accurate results.

Bayes Theorem of Conditional Probability

Before we dive into Bayes theorem, let's review marginal, joint, and conditional probability.

- Recall that marginal probability is the probability of an event, irrespective of other random variables. If the random variable is independent, then it is the probability of the event directly, otherwise, if the variable is dependent upon other variables, then the marginal probability is the probability of the event summed over all outcomes for the dependent variables, called the sum rule.

  - **Marginal Probability**: The probability of an event irrespective of the outcomes of other random variables, e.g. P(A).

    The joint probability is the probability of two (or more) simultaneous events, often described in terms of events A and B from two dependent random variables, e.g. X and Y. The joint probability is often summarized as just the outcomes, e.g. A and B.

  - **Joint Probability**: Probability of two (or more) simultaneous events, e.g. P(A and B) or P(A, B).

TIME SERIES ANALYSIS

- Time-series analysis is a popular machine-learning technique for sales forecasting. It involves analysing data collected over time, such as sales data, and modelling historical trends and patterns to make predictions about the future.
- Time-series analysis is useful for predicting sales because it takes into account historical patterns and trends, such as seasonality and long-term trends, to make accurate predictions.
- The technique can also identify anomalies or unexpected events that may impact future sales, allowing businesses to make necessary adjustments to their strategy.

Machine learning serves as the ultimate booster here, allowing to better handle:

- **Stock prices forecasting** — the data on the history of a stock price forecasting project is combined with the data on both regular and irregular stock market spikes and drops and can be used to gain insightful predictions of the most probable upcoming stock price shifts.

- **Demand and sales forecasting** — customer behavior patterns data along with inputs from the history of purchases, timeline of demand, seasonal impact, etc., enable ML models to point out the most potentially demanded products and hit the spot in the dynamic market.

- **Web traffic forecasting** — common data on usual traffic rates among competitor websites is bunched up with input data on traffic-related patterns in order to predict web traffic rates during certain periods.

- **Climate and weather prediction** — time-based data is regularly gathered from numerous interconnected weather stations worldwide, while ML techniques allow thorough analysis and interpretation for future forecasts based on statistical dynamics.

- **Demographic and economic forecasting** — there are tons of statistical inputs in demographics and economics, which are most efficiently used for ML-based time-series predictions. As a result, the most fitting target audience can be picked and the most efficient ways to interact with that particular TA can be elaborated.

- **Scientific studies forecasting** — Machine Learning and deep learning for time series forecasting accelerate the rates of polishing up and introducing scientific innovations dramatically. For instance, science data that requires an indefinite number of analytical iterations can be processed much faster with the help of patterns automated by machine learning.

Classic Methods of Time Series Forecasting

- Multi-Layer Perceptron (MLP)
- ARIMA
- Bayesian Neural Network (BNN)
- Radial Basis Functions Neural Network (RBFNN)
- Kernel Regression or Generalized Regression Neural Network (GRNN)
- K-Nearest Neighbor Regression Neural Network (KNN)
- CART Regression Trees (CART)
- Support Vector Regression (SVR)
- Gaussian Processes (GP)

RECOMMENDER SYSTEMS

- Recommender systems are one of the most successful and widespread application of machine learning technologies in business.
- You can find large scale recommender systems in retail, video on demand, or music streaming.
- Machine learning algorithms in recommender systems are typically classified into two categories — content based and collaborative filtering methods although modern recommenders combine both approaches. Content based methods are based on similarity of item attributes and collaborative methods calculate similarity from interactions.

1. Frequent item sets, also known as association rules, are a fundamental concept in association rule mining, which is a technique used in data mining to discover relationships between items in a dataset. The goal of association rule mining is to identify relationships between items in a dataset that occur frequently together.

2. A frequent item set is a set of items that occur together frequently in a dataset. The frequency of an item set is measured by the support count, which is the number of transactions or records in the dataset that contain the item set. For example, if a dataset contains 100 transactions and the item set {milk, bread} appears in 20 of those transactions, the support count for {milk, bread} is 20.

3. Association rule mining algorithms, such as Apriori or FP-Growth, are used to find frequent item sets and generate association rules. These algorithms work by iteratively generating candidate item sets and pruning those that do not meet the minimum support threshold. Once the frequent item sets are found, association rules can be generated by using the concept of confidence, which is the ratio of the number of transactions that contain the item set and the number of transactions that contain the antecedent (left-hand side) of the rule.

4. Frequent item sets and association rules can be used for a variety of tasks such as market basket analysis, cross-selling and recommendation systems. However, it should be noted that association rule mining can generate a large number of rules, many of which may be irrelevant or uninteresting. Therefore, it is important to use appropriate measures such as lift and conviction to evaluate the interestingness of the generated rules.

   - **Association Mining** searches for frequent items in the data set. In frequent mining usually, interesting associations and correlations between item sets in transactional and relational databases are found. In short, Frequent Mining shows which items appear together in a transaction or relationship.

   - **Need of Association Mining:** Frequent mining is the generation of association rules from a Transactional Dataset. If there are 2 items X and Y purchased frequently then it's good to put them together in stores or provide some discount offer on one item on purchase of another item. This can really increase sales. For example, it is likely to find that if a customer buys **Milk** and **bread** he/she also buys **Butter**. So the association rule is **['milk]^['bread']=>['butter']**. So the seller can suggest the customer buy butter if he/she buys Milk and Bread.